

LogPulse AI - Incident Report

Generated : 2026-06-01 13:05:58 UTC

Trace ID : 8f3fe2d9-7c67-469d-863f-37bf8daf0dff

1 - Incident Summary

Incident ID : 8f3fe2d9
Category : Memory
Severity : SeverityLevel.FATAL
Source : Kubernetes
Cache : MISS

2 - Raw Log

2026-01-15T22:14:07Z FATAL kubelet: Pod worker-api-7d9f8b-xk2q OOMKilled. Container exceeded memory limit of 256Mi. Exit code 137. Restart count: 14. Node: worker-1. Namespace: production.

3 - Classification

Category : Memory
Severity : SeverityLevel.FATAL
Summary : Kubernetes pod killed due to OOMKilled container exceeding its memory limit.
Source : Kubernetes

4 - Diagnosis

Root Cause : The container's configured memory limit of 256Mi is insufficient for the application's actual memory consumption, causing Kubernetes to repeatedly OOMKill the pod.
Confidence : 95%
Reasoning : The raw log explicitly states 'OOMKilled. Container exceeded memory limit of 256Mi' with a high restart count, and the retrieved playbook is titled 'Pod OOMKilled: Insufficient Memory Limit' with a resolution to increase memory limits.
Source Tier : Ilm
Used History : No
Used Playbook : Yes

5 - Proposed Solution

Explanation:

The container in pod worker-api-7d9f8b-xk2q was terminated with exit code 137 because its memory usage exceeded the configured limit of 256Mi.

Steps:

1. Retrieve the detailed configuration and termination state of the OOMKilled pod.
2. Retrieve the logs from the previous crashed container instance to investigate the cause of the memory spike.
3. Increase the memory limit to 512Mi and request to 256Mi for the deployment to prevent further OOM kills.
4. Track the deployment rollout to ensure the new resource limits are applied successfully.

Commands:

```
$ kubectl get pod worker-api-7d9f8b-xk2q -n production -o yaml
$ kubectl logs worker-api-7d9f8b-xk2q -n production --previous --tail=100
$ kubectl set resources deployment/worker-api -n production --limits=memory=512Mi
--requests=memory=256Mi
$ kubectl rollout status deployment/worker-api -n production
```

6 - Risk Assessment

Risk Level : **RISKLEVEL.SAFE**

Safe Commands (4):

```
[OK] kubectl get pod worker-api-7d9f8b-xk2q -n production -o yaml
[OK] kubectl logs worker-api-7d9f8b-xk2q -n production --previous --tail=100
[OK] kubectl set resources deployment/worker-api -n production --limits=memory=512Mi --requests=memory=256Mi
[OK] kubectl rollout status deployment/worker-api -n production
```

Flagged Commands (0):

(none)

7 - Operator Decision

Operator rejected -- no execution.

8 - Historical Context

Times Seen : 1

First occurrence -- no prior incidents.

9 - Incident Timeline

```
[09:59:55] INCIDENT_RECEIVED -> Incident received (trace_id=8f3fe2d9...)
[09:59:55] CACHE_MISS -> Cache miss -- routing to classifier
[10:00:04] INCIDENT_CLASSIFIED -> Memory / Fatal
[10:00:07] RAG_RETRIEVED -> Playbook retrieved (filtered) -- Memory conf=0.82
[10:00:19] ROOT_CAUSE_GENERATED -> The container's configured memory limit of 256Mi is
insufficient for the application's actual memory consumption, causin
[10:00:56] SOLUTION_GENERATED -> 4 step(s), 4 command(s)
[10:00:56] RISK_ASSESSED -> safe -- 0 flagged
[10:05:56] WORKFLOW_COMPLETED -> Completed in 361694ms
```